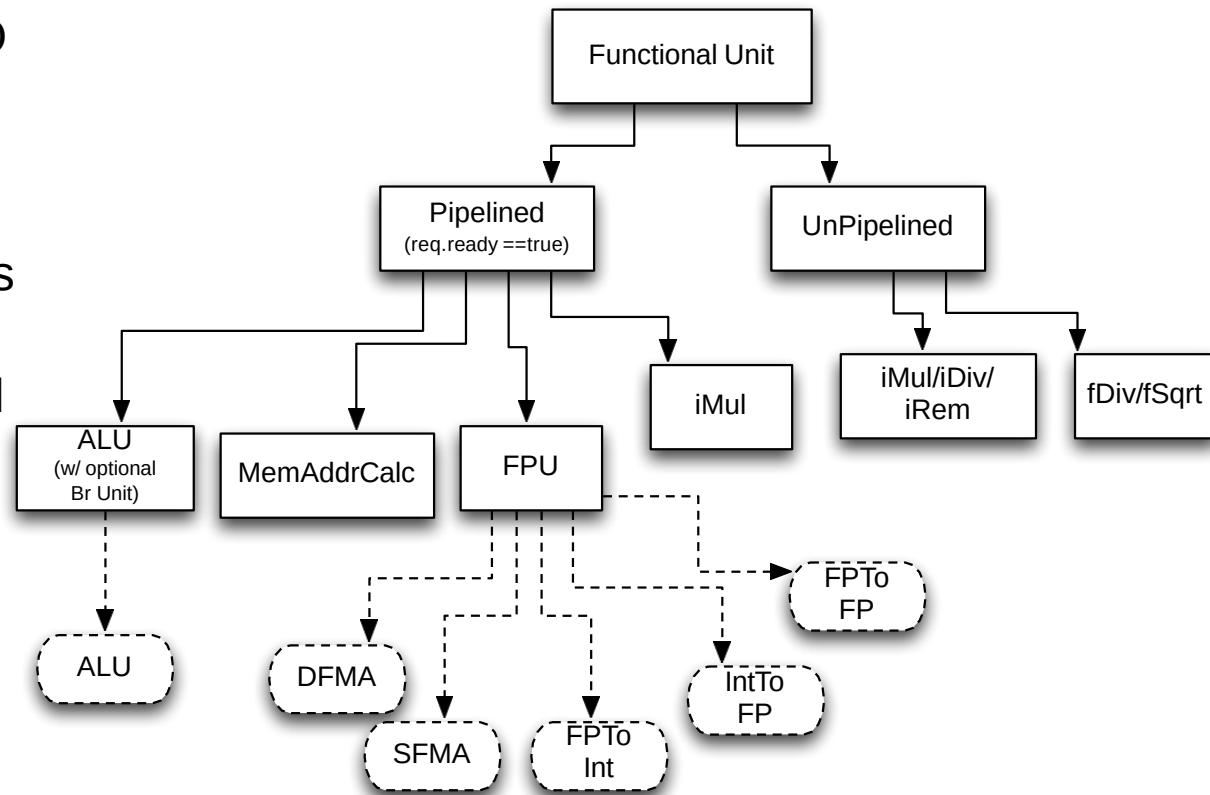


Open--source Berkeley RISC-V

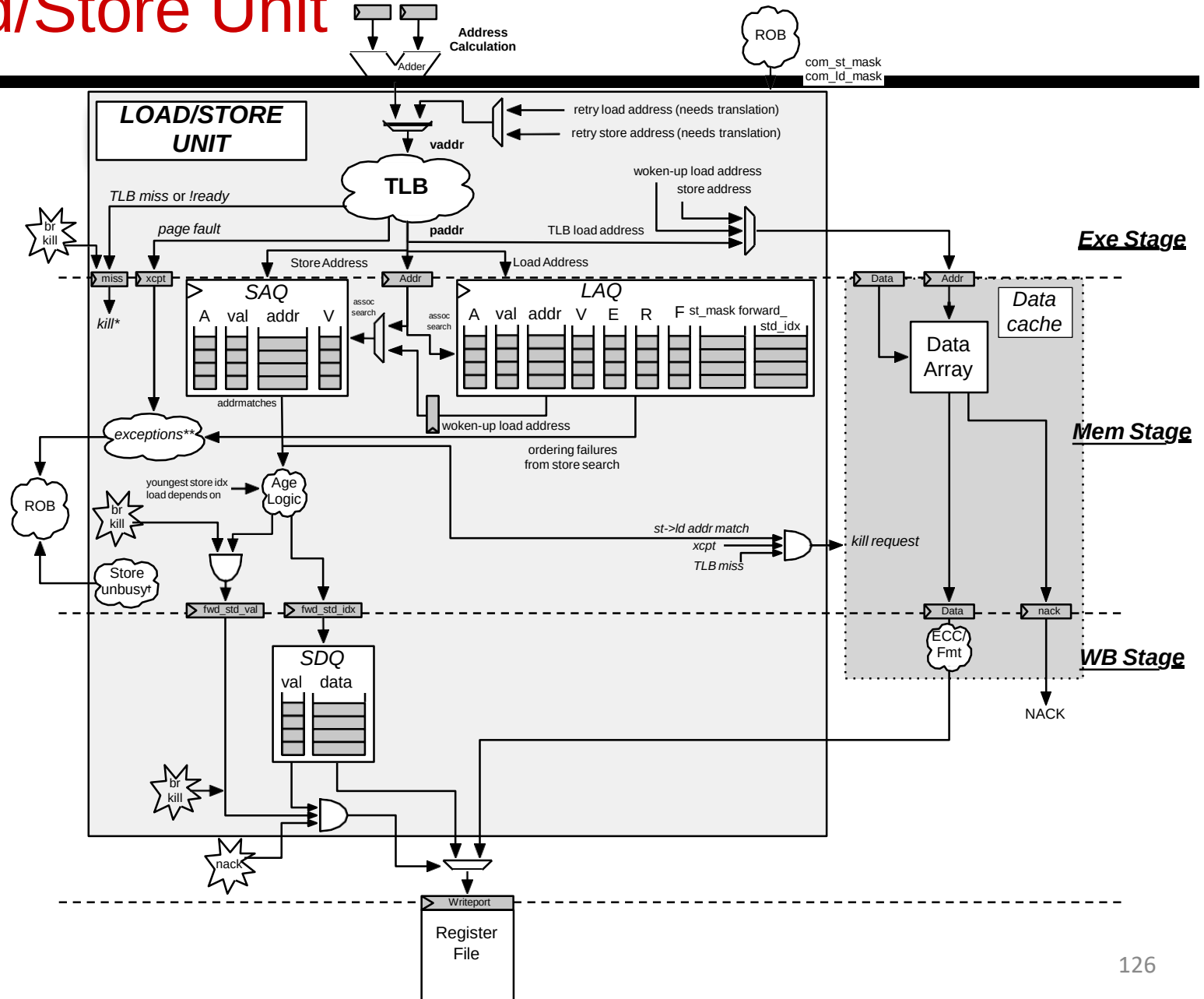
Processors

- Abstract FunctionalUnit
 - describes common IO
- Pipelined/Unpipelined
 - handles storing uop metadata, branch resolution, branch kills
- Concrete Subclasses
 - instantiates the actual expert-written FU
 - no modifications required to get FU working with speculative OoO
 - allows easy “stealing” of external code



TSMC 45nm floorplan

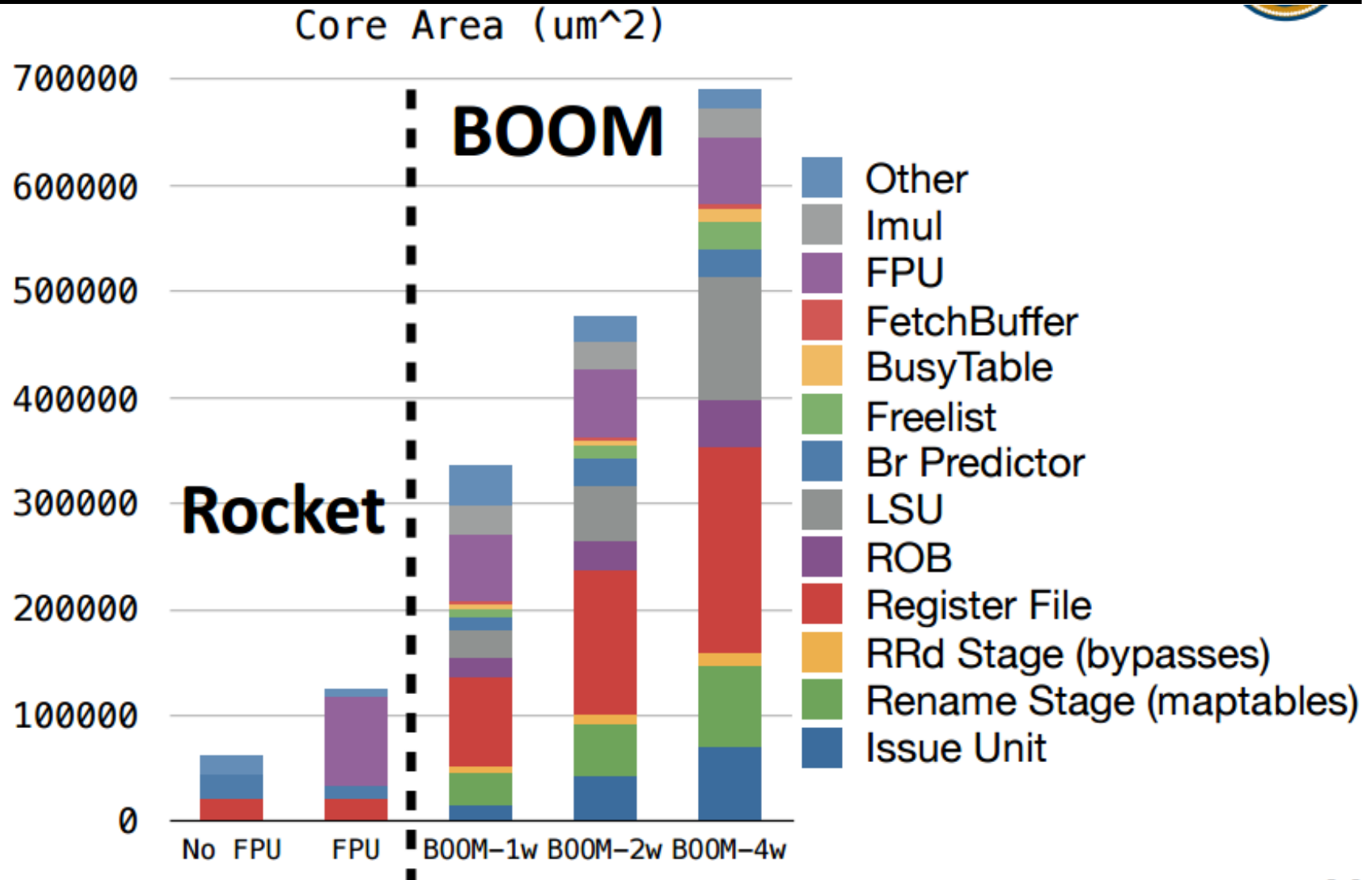
Load/Store Unit



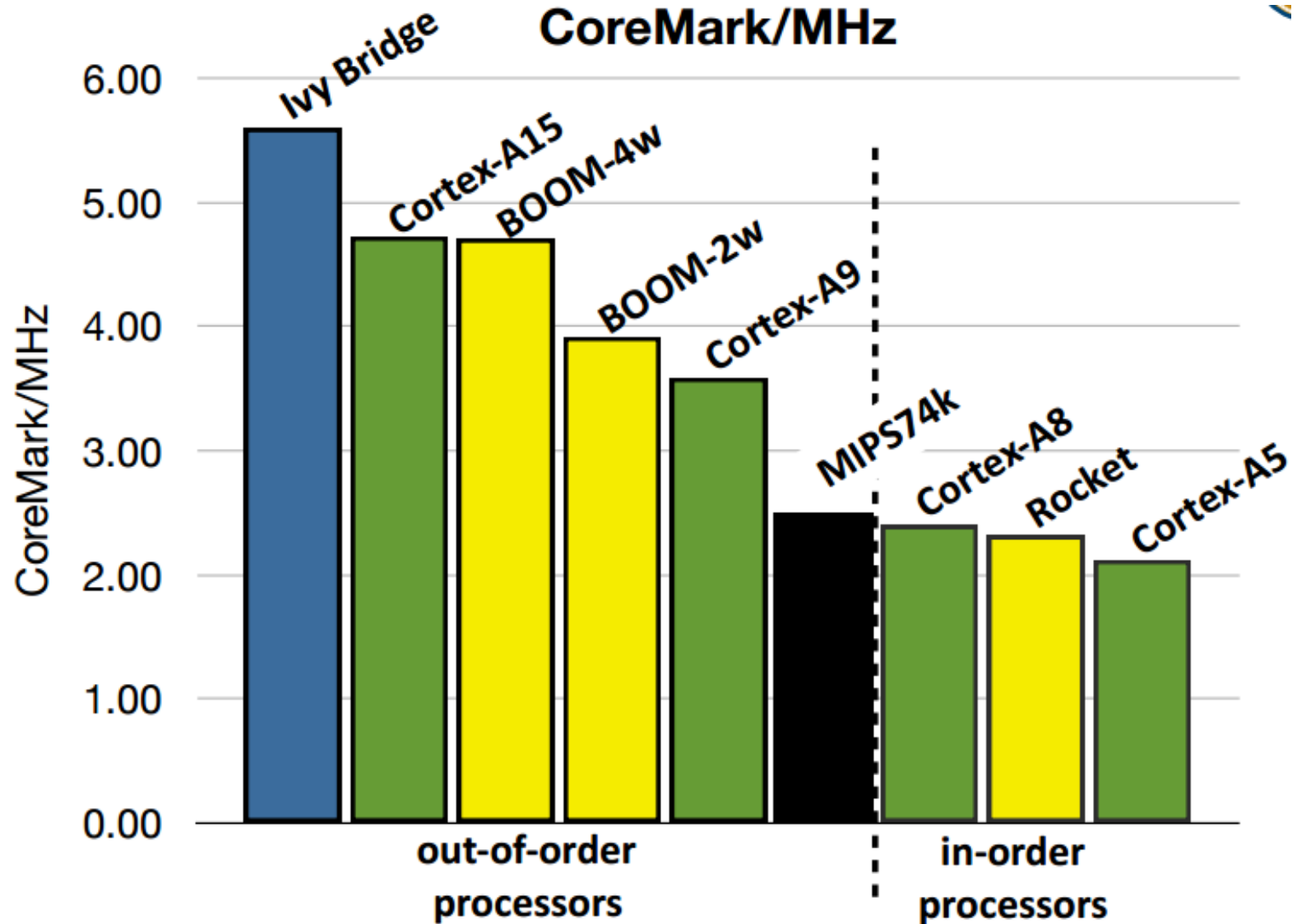
Question of the Day

- How many in-order cores do you think take up the same area as an out-of-order core?

How Many In-Order In The Same Area?



Performance?

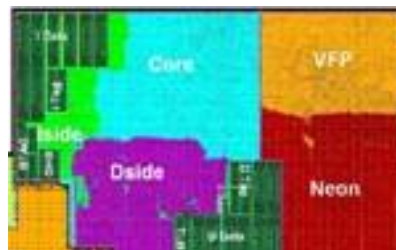


Comparison against ARM

Category	ARM Cortex-A9	RISC-V BOOM-2w
ISA	32-bit ARM v7	64-bit RISC-V v2 (RV64G)
Architecture	2 wide, 3+1 issue Out-of-Order 8-stage	2 wide, 3 issue Out-of-Order 6-stage
Performance	3.59 CoreMarks/MHz	3.91 CoreMarks/MHz
Process	TSMC 40GPLUS	TSMC 40GPLUS
Area with 32K caches	~2.5 mm ²	~1.00 mm ²
Area efficiency	1.4 CoreMarks/MHz/mm ²	3.9 CoreMarks/MHz/mm ²
Frequency	1.4 GHz	1.5 GHz
Power	0.5-1.9 W (2 cores + L2) @ TSMC 40nm, 0.8-2.0 GHz	0.25 W (1 core + L1) @ TSMC 45nm, 1 GHz

+9%!

note:
not to scale



Repositories

- BOOM
 - <https://github.com/ucb-bar/riscv-boom>
 - just the BOOM core code
- Rocket-chip
 - <https://github.com/ucb-bar/rocket-chip>
 - the rest of the SoC
 - chisel, rocket, uncore, junctions, riscv-tools, fpga-zynq, etc.
 - generates C++ emulator, verilog for FPGA
 - <http://riscv.org/tutorial-hpca2015/riscv-rocket-chip-tutorial-bootcamp-hpca2015.pdf> (for more information)

BOOM Quick-start

```
$ export ROCKETCHIP_ADDONS="boom"
$ git clone https://github.com/ucb-bar/rocket-chip.git
$ cd rocket-chip
$ git checkout boom
$ git submodule update --init
$ cd riscv-tools
$ git submodule update --init --recursive riscv-tests
$ cd ../emulator; make run CONFIG=BOOMCPPConfig
```

- "BOOM-chip" is (currently) a branch of rocket-chip
- "make run" builds and runs riscv-tests suite
- there are many different CONFIGs available!
 - rocket-chip/src/main/scala/PrivateConfigs.scala
 - rocket-chip/boom/src/main/scala/configs.scala
 - slightly different configs for different targets (CPP, FPGA, VLSI)

How do you verify and debug BOOM?

- parameterization makes this tough!
- tested with:
 - riscv-tests
 - (assembly functional tests + bare-metal micro-benchmarks)
 - CoreMark + riscv-pk
 - SPEC + Linux
 - riscv-torture

riscv-torture

- now open-source!
 - <https://github.com/ucb-bar/riscv-torture>
- torture generates random tests to stress the core pipeline
- runs test on Spike and your processor
 - architectural register state dumped to memory on program termination
 - diff state
 - if error found, finds smallest version of test that exhibits an error

riscv-torture quick-start

Run Rocket-chip (BOOM-chip?)

```
$ git clone https://github.com/ucb-bar/rocket-chip.git
$ cd rocket-chip
$ git submodule update --init
$ cd riscv-tools
$ git submodule update --init --recursive riscv-tests
$ cd ../emulator; make run CONFIG=BOOMCPPConfig
```

Run Torture

```
$ cd rocket-chip
$ git clone https://github.com/ucb-bar/riscv-torture.git
$ cd riscv-torture
$ git submodule update --init
$ vim Makefile # change RTL_CONFIG=BOOMCPPConfig
$ make igentest # test that torture works, gen a single test
$ make cnight # run C++ emulator overnight
```

Commit Logging

- BOOM can generate commit logs
 - (priv level, PC, inst, wb raddr, wb data)
- Spike supports generating commit logs!
 - configure Spike with "--enable-commitlog" flag
 - outputs to stderr
- only semi-automated
 - valid differences from Spike and hw
 - e.g., spinning on LR+SC

modify the build.sh in riscv-tools

```
build_project riscv-isa-sim --prefix=$RISCV --with-fesvr=$RISCV --enable-commitlog
```

What documentation is available?

- A design document is in progress
 - <https://github.com/ccelio/riscv-boom-doc>
- Wiki
 - <https://github.com/ucb-bar/riscv-boom/wiki>

Conclusion

- BOOM is RV64G, runs SPEC on Linux on an FPGA
- BOOM is ~10k loc and 4 person-years of work
- excellent platform for prototyping new ideas
- now open-source!

